

Lecture 2 2026-09-03

Today: linear algebra "crash course",
gradient descent

Vector norms: assign "size" to a vector
that we will later want to minimize

properties: $\|x\|_p$ (p-norm)

1. $\|x\|_p \geq 0 \quad \forall x \in \mathbb{R}^n$
and $\|x\|_p = 0$ if and only if $x = 0$

2. $\|cx\|_p = |c| \|x\|_p$ for $c \in \mathbb{R}$

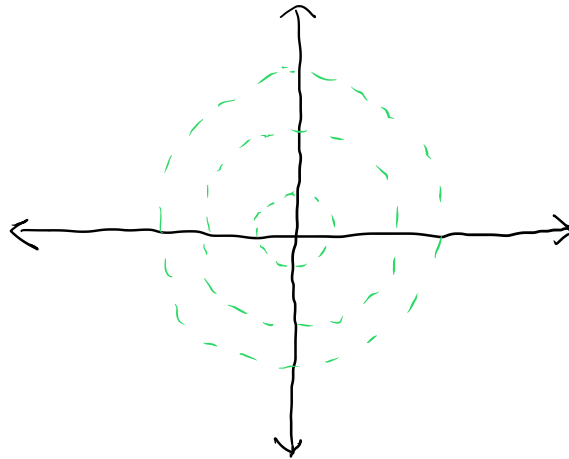
3. $\|x+y\|_p \leq \|x\|_p + \|y\|_p$

p-norm $\|x\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}$

2-norm: $\|x\|_2 = \sqrt{\sum_{i=1}^n (x_i)^2}$

colloquially: "as the crow flies"

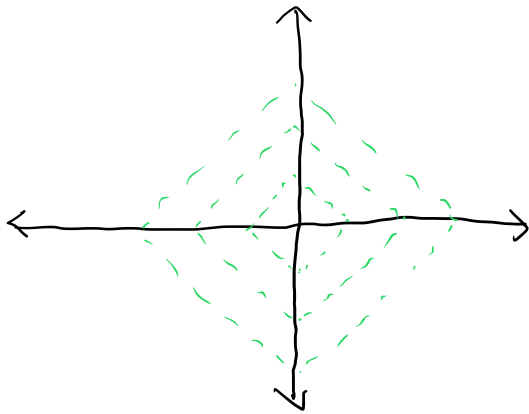
level sets look like:



1-norm: $\|x\|_1 = \sum_{i=1}^n |x_i|$

colloquially: Manhattan distance or
"taxi cab" norm

level curves look like:



l_1 -norm is known as "sparsity promoting"

i.e., minimize $\|x\|_1$

often yields more 0-elements in the optimal solution.

→ useful property in e.g., signal processing, ML

Property: if p, q are integers such that

$0 < p < q$, then

$$\|x\|_p \geq \|x\|_q \text{ for all } x$$

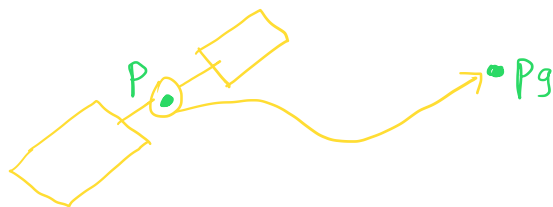
Why do we care?

Later on, we'll use the $\|x\|_p$ norm to minimize useful physical properties

e.g., if a spacecraft position is described as $p = (p_x, p_y) \in \mathbb{R}^2$, then

$$\underset{p}{\text{minimize}} \|p - p_g\|_p$$

will minimize position with respect to some desired goal p_g



Eigenvalues

If A is a square matrix

$A \in \mathbb{R}^{n \times n}$, then eigenvalue λ

$$Ax = \lambda v$$

Solve for eigenvalue via its
characteristic equation:

$$\det |A - \lambda I| = 0$$

Eigenvalue decomposition

Let $v_1, \dots, v_n$ be n linearly independent eigenvectors of A .

Then we can express $A = T \Lambda T^{-1}$

where $T = \begin{pmatrix} v_1 & v_2 & \dots & v_n \end{pmatrix}$

$$\Lambda = \begin{pmatrix} \lambda_1 & & & 0 \\ & \lambda_2 & & \\ & & \ddots & \\ 0 & & & \lambda_n \end{pmatrix}$$

Computational complexity

How does the runtime of an algorithm scale with respect to the size n with respect to its argument/input?

→ The choice and performance of the optimization algorithm we will use is determined by computational complexity

Common to consider big- O notation:

$O(\cdot)$ input size n

examples:

$O(n)$: linear time

adding two vectors of size n

```

def add(x, y):
    sum = (0, ..., 0)

    for i = 1 → n
        sumi = xi + yi

    return sum

```

matrix multiplication $y = Ax$ $A \in \mathbb{R}^{m \times n}$

$$\begin{bmatrix} \text{---} a_1 \text{---} \\ \vdots \\ \text{---} a_m \text{---} \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}$$

```

def multiply(A, x):
    y = (0, ..., 0)

    for i = 1 → m
        yi = 0
        for j = 1 → n

```

$y_i += A_{ij} x_j$
return y

Matrix inverse: A is square and invertible

$$A \in \mathbb{R}^{n \times n}$$

→ complexity of A^{-1} is $\mathcal{O}(n^3)$

→ "optimized" routines used in high-performance

linear algebra libraries get down to

$$\mathcal{O}(n^w) \text{ where } w \approx 2.40$$

Unconstrained optimization

We will study problems of the form:

$$\min_{x \in \mathbb{R}^n} f(x)$$

$$\text{subj. to: } \begin{aligned} f_i(x) &\leq 0 & i = 1, \dots, n_{\text{ineq}} \\ h_i(x) &= 0 & i = 1, \dots, n_{\text{eq}} \end{aligned}$$

Gradient:

Let $f(x)$ such that $f: \mathbb{R}^n \rightarrow \mathbb{R}$
and $f \in \mathcal{C}^1$

$$\rightarrow \text{gradient } \nabla f(x) = \begin{pmatrix} \partial f / \partial x_1 \\ \vdots \\ \partial f / \partial x_n \end{pmatrix}$$

Hessian:

Let $f: \mathbb{R}^n \rightarrow \mathbb{R}$ and $f \in \mathcal{C}^2$

$$\text{then Hessian } \nabla_{xx} f(x) = \begin{pmatrix} \partial^2 f / \partial x_1^2 & \dots & \partial^2 f / \partial x_1 \partial x_n \\ \vdots & \ddots & \vdots \\ \partial^2 f / \partial x_n \partial x_1 & \dots & \partial^2 f / \partial x_n^2 \end{pmatrix}$$

H is a symmetric matrix and $H \in \mathbb{R}^{n \times n}$

The gradients and Hessians will be useful when we want to construct a local approximation of $f(x)$ using Taylor series

recall scalar version when $f: \mathbb{R} \rightarrow \mathbb{R}$

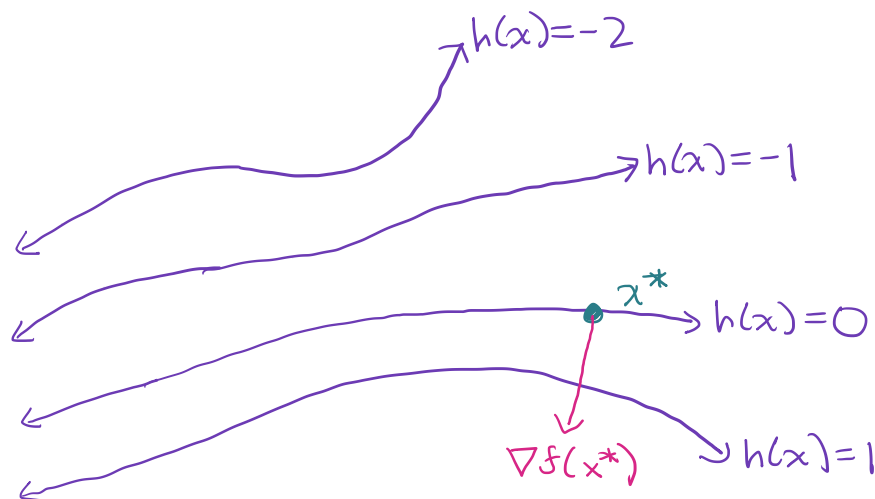
$$\rightarrow f(x) \approx f(\bar{x}) + f'(\bar{x})(x - \bar{x}) + \frac{1}{2} f''(\bar{x})(x - \bar{x})^2$$

in vector version for $f: \mathbb{R}^n \rightarrow \mathbb{R}$

$$\begin{aligned} \rightarrow f(x) \approx f(\bar{x}) + \nabla f(\bar{x})^T (x - \bar{x}) \\ + \frac{1}{2} (x - \bar{x})^T \nabla_{xx} f(\bar{x}) (x - \bar{x}) \end{aligned}$$

What information about a function does the gradient $\nabla f(x)$ provide?

Consider the level curves of some function $f(x)$.



A gradient points **orthogonal** to the level curve and in the direction of increasing value (low-to-high).

Key insight: if we want to minimize $f(x)$, then $\nabla f(x)$ can provide us information about

then $\nabla f(x)$ can provide us information about direction of increase (and hence decrease)

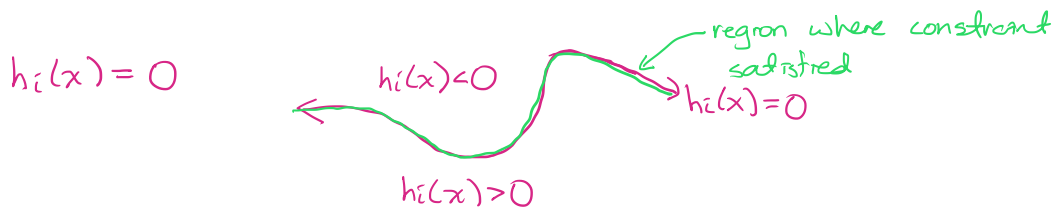
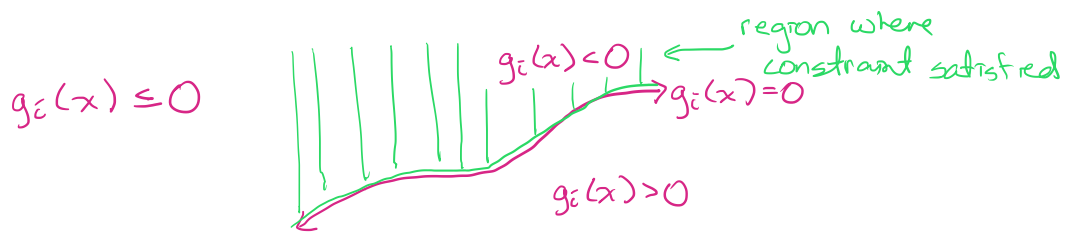
Feasible set for problem $\mathcal{P}$:

$$\mathcal{P}: \min_x f(x)$$

$$\text{subj. to: } g_i(x) \leq 0 \quad i = 1, \dots, n_{\text{ineq}}$$

$$h_c(x) = 0 \quad c = 1, \dots, n_{\text{eq}}$$

g_i is an inequality that maps $x \in \mathbb{R}^n \rightarrow g_i(x) \in \mathbb{R}$



Feasible set for $\mathcal{P}$:

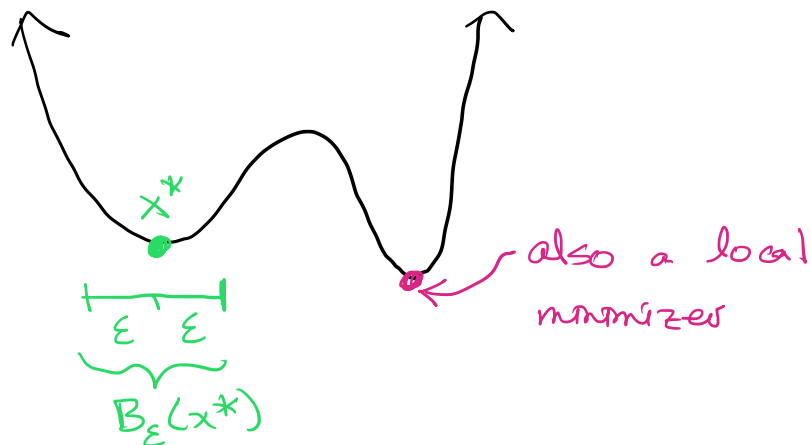
$$\Omega = \{ x \in \mathbb{R}^n \mid g_i(x) \leq 0, \quad i=1, \dots, n_{\text{ineq}} \\ \text{and } h_i(x) = 0, \quad i=1, \dots, n_{\text{eq}} \}$$

$\rightarrow \mathcal{P}$ can be expressed as:

$$\begin{aligned} \min_x \quad & F(x) \\ \text{subj. to: } & x \in \Omega \end{aligned}$$

Local minimizer: a point $x^* \in \Omega$ is a local minimizer of $\mathcal{P}$ if $\exists B_\varepsilon(x^*)$ for $\varepsilon > 0$ such that $\forall y \in B_\varepsilon(x^*) \cap \Omega$, $f(y) \geq f(x^*)$

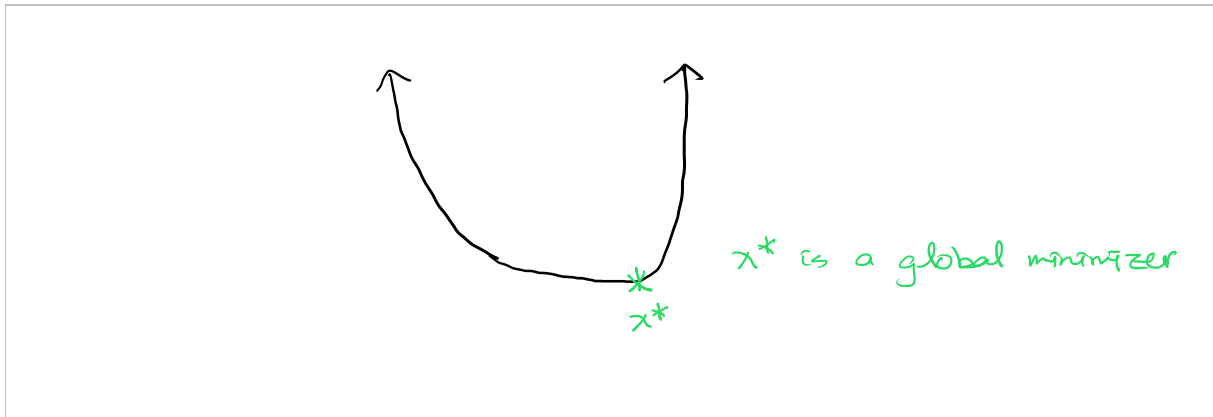
Epsilon ball: $B_\varepsilon(x) = \{y \mid d(x, y) \leq \varepsilon\}$



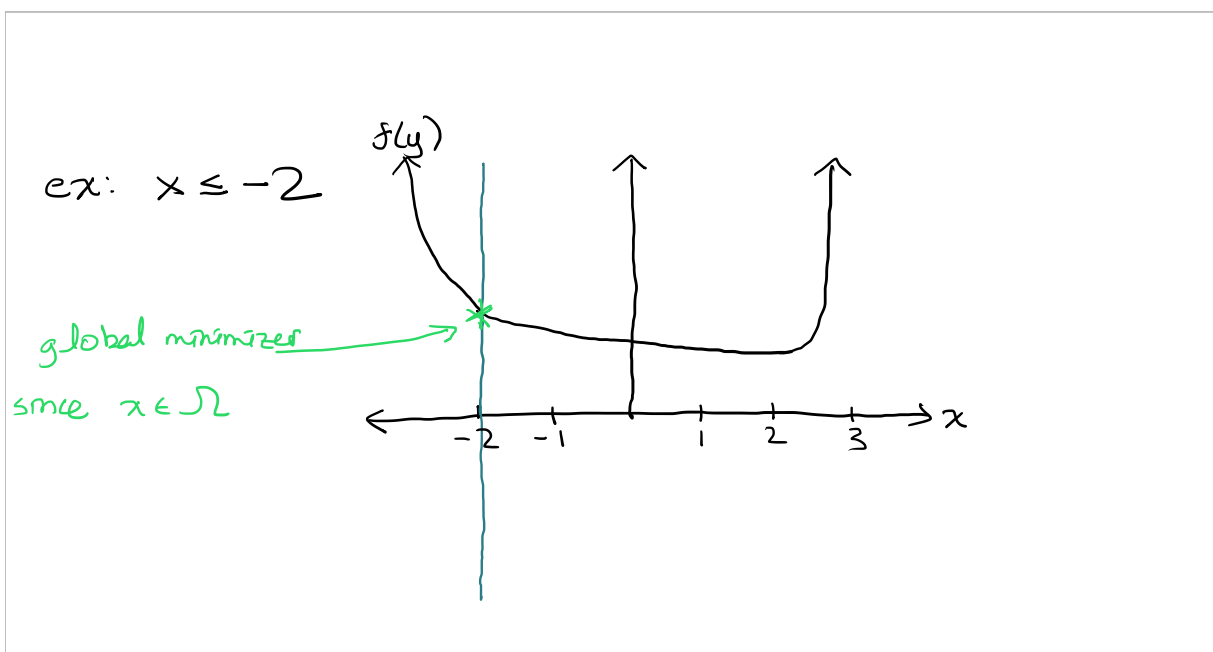
Since $f(y) \geq f(x^*) \forall y \in B_\varepsilon(x^*)$, x^* is a local minimizer

Global minimizer: $x^* \in \Omega$ is a global minimizer

of $\mathcal{P}$ if $\forall y \in \Omega, f(y) \geq f(x^*)$



Now consider constraints:



Let's consider smooth and unconstrained optimization

$$\min_x f(x)$$

→ Applying the definition of a local minimizer, we recover necessary conditions of optimality:

First-order necessary conditions of optimality

(NOC): if x^* is a local minimizer of $f(x) \rightarrow \nabla_x f(x^*) = 0$

Second-order NOC:

if x^* is a local minimizer of $f(x) \rightarrow \nabla_{xx}^2 f(x^*) \succeq 0$

Gradient descent: iterative algorithm to find

local minimizer satisfying 1st-order NOC

Want to find x^* such that $\nabla f(x^*) = 0$

1. Start with some point $x^{(0)}$ ← superscript denotes iterate

2. for $k = 1 \rightarrow N_{\max}$

$$d^{(k)} = -\nabla_x f(x^{(k)}) \quad // \text{descent direction}$$

$$x^{(k+1)} = x^{(k)} + \alpha d^{(k)}$$

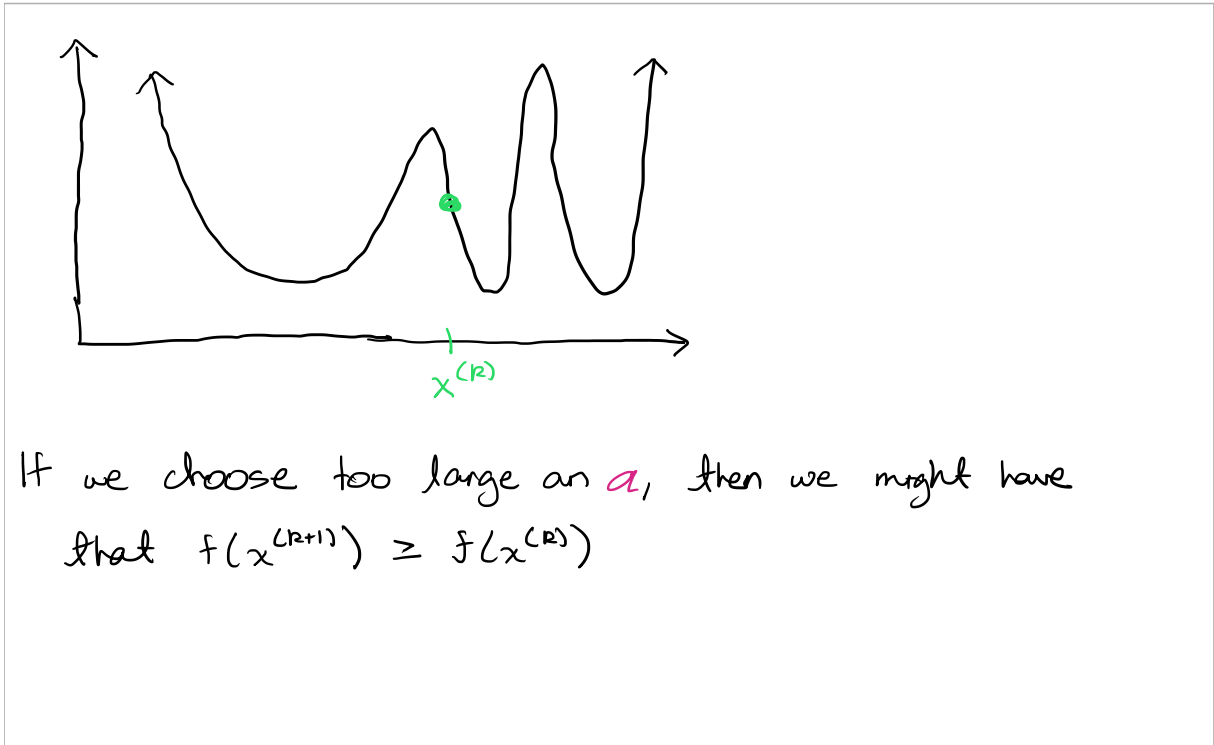
if $\|d^{(k)}\|_2 = 0$:
terminate

In practice, we define some numerical tolerance ϵ_{tol} to account for floating point error. i.e.,
if $\|d^{(k)}\|_2 \leq \epsilon_{tol}$

Note: we take step in direction $d^{(k)} = -\nabla_x f(x^{(k)})$

α : step size or learning rate

Step size α : determines how far to move in descent direction



If we choose too large an α , then we might have that $f(x^{(k+1)}) \geq f(x^{(k)})$

Backtracking line search: pick largest α such that $f(x^{(k)} - \alpha g^{(k)}) \leq f(x^{(k)})$

Line Search($f, x^{(k)}, g^{(k)}, a, b$): $\leftarrow \alpha > b$

for α in range(a, b):

if $f(x^{(k)} - \alpha g^{(k)}) \leq f(x^{(k)})$:

$g^{(k)} \leq \nabla f(x^{(k)});$
return α

Two stronger alternatives include:

Wolfe conditions: if $d^{(k)}$ is descent direction:

1. $f(x^{(k)} + \alpha d^{(k)}) \leq f(x^{(k)}) + c, \alpha d^{(k)T} \nabla f(x^{(k)})$

(Armijo condition)

$$2. -d^{(k)T} \nabla f(x^{(k)} + \alpha d^{(k)}) \leq -c_2 d^{(k)T} \nabla f(x^{(k)}) \quad (\text{curvature condition})$$

Strong Wolfe conditions: includes preceding two conditions (Armijo + curvature conditions) and adds

$$3. |d^{(k)T} \nabla_x f(x^{(k)} + \alpha d^{(k)})| \leq c_2 |d^{(k)T} \nabla f(x^{(k)})|$$

Newton method: use the "optimal" step size given from 2nd-order Taylor series expansion

2nd-order Taylor series expansion about $\bar{x}$:

$$f(\bar{x} + \delta x) \approx f(\bar{x}) + \nabla_x f(\bar{x})^T \delta x + \frac{1}{2} \delta x^T \nabla_{xx}^2 f(\bar{x}) \delta x + \text{H.O.T.}$$

Impose 1st-order NOC:

$$\nabla_{\delta x} f(\bar{x} + \delta x) = \nabla_x f(\bar{x}) + \nabla_{xx}^2 f(\bar{x}) \delta x := 0$$

$$\Rightarrow \delta x = - \underbrace{(\nabla_{xx}^2 f(\bar{x}))^{-1}}_{\text{"optimal" } \alpha} \nabla_x f(\bar{x})$$

Newton method pseudo-code:

1. Start with $x^{(0)}$

2. for $k = 1 \rightarrow N_{\max}$

$$g^{(k)} = \nabla_x f(x^{(k)})$$

$$H^{(k)} = \nabla_{xx} f(x^{(k)})$$

$$x^{(k+1)} = x^{(k)} - (H^{(k)})^{-1} g^{(k)}$$

$$\text{if } \|g^{(k)}\|_2 \leq \epsilon_{\text{tol}}$$

terminate

Advantages of Newton method: converges in fewer steps given some precision ϵ compared to GD

Disadvantages: requires inverse of Hessian at each iteration ($\mathcal{O}(n^3)$ complexity)

Asides:

1. There exist "hybrid" methods

BFGS: a quasi-Newton method with complexity

$\mathcal{O}(n^2)$ that approximates the Hessian

2. If the Hessian is ill-conditioned (i.e., matrix

inverse is difficult to compute), then can pad

it i.e., $(H + \underset{\uparrow}{\lambda} I)^{-1}$

λ is some positive value